arXiv:2605.09630v1 [cs.CL] 10 May 2026

Scratchpad Patching: Decoupling Compute from Patch Size in Byte-Level Language Models

Lin Zheng[1,2]* **Vasilisa Bashlovkina**[1] **Timothy Dozat**[1]
Dan Garrette[1] **Laura Rimell**[1] **Joshua Maynez**[1]
[1]Google DeepMind [2]The University of Hong Kong
linzhengs@google.com

Abstract

Tokenizer-free language models eliminate the tokenizer step of the language modeling pipeline by operating directly on bytes; patch-based variants further aggregate contiguous byte spans into patches for efficiency. However, the average patch size chosen at the model design stage governs a tight trade-off: larger patches reduce compute and KV-cache footprint, but degrade modeling quality. We trace this trade-off to *patch lag*: until a patch is fully observed, byte predictions within it must rely on a stale representation from the previous patch to preserve causality; this lag widens as patches grow larger. We introduce *Scratchpad Patching* (SP), which inserts transient scratchpads inside each patch to aggregate the bytes seen so far and refresh patch-level context for subsequent predictions. SP triggers scratchpads using next-byte prediction entropy, selectively allocating compute to information-dense regions and enabling post-hoc adjustment of inference-time compute. Across experiments on natural language and code, SP improves model quality at the same patch size; for example, even at 16 bytes per patch, SP-augmented models match or closely approach the byte-level baseline on downstream evaluations while using a $16\times$ smaller KV cache over patches and 3–$4\times$ less inference compute.

1 Introduction

Modern language models rely on tokenization [Sennrich et al., 2016, Kudo and Richardson, 2018] to derive input representations and segment text into shorter token sequences. This handcrafted, non-end-to-end process introduces distinct drawbacks: the sequence shortening achieved by a fixed tokenizer is difficult to adapt or scale [Yu et al., 2025], the model is sensitive to prompt formatting [Microsoft, 2023, Lundberg and Ribeiro, 2023], and glitch tokens can disrupt inference [Rumbelow and Watkins, 2023, Land and Bartolo, 2024, Yang et al., 2024]. Recent research has therefore pivoted toward *tokenizer-free* modeling—methods that operate directly on bytes without an externally defined subword vocabulary [Sutskever et al., 2011, Graves, 2013, Radford et al., 2017, Chung et al., 2017, Hwang and Sung, 2017, Al-Rfou et al., 2019, Choe et al., 2019, Xue et al., 2022, Clark et al., 2022, Wang et al., 2024, Zheng et al., 2025]. To mitigate the prohibitive cost of long byte sequences, *patch-based* tokenizer-free models (Section 2) aggregate contiguous bytes into higher-level *patches*, shortening the effective sequence length [Clark et al., 2022, Nawrot et al., 2022, Tay et al., 2022, Yu et al., 2023, Nawrot et al., 2023, Slagle, 2024, Ahia et al., 2024, Pagnoni et al., 2024, Neitemeier et al., 2025, Owodunni et al., 2025, Videau et al., 2025, Hwang et al., 2025, Minixhofer et al., 2025].

While promising, the standard approach to segmentation and formation of patch representations introduces a tight trade-off. Larger patch sizes yield fewer patches per input, improving computational efficiency and reducing KV-cache usage, but they also update patch-level context less frequently, forcing more byte predictions to be made from stale patch-level context. We call this staleness

*Work done during an internship at Google DeepMind.

Preprint.

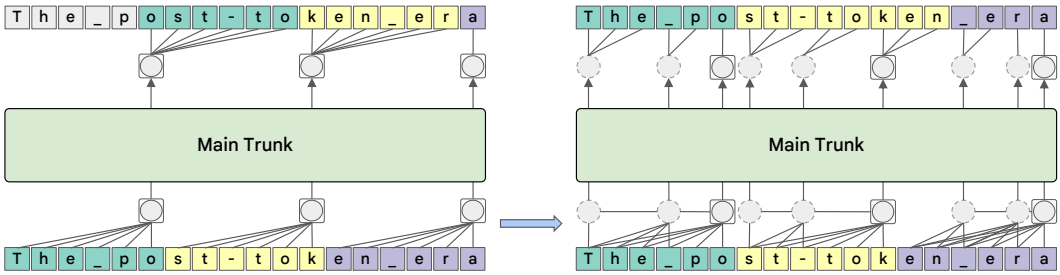


Figure 1: **Scratchpad Patching (SP).** *Left:* a standard patch-based byte-level model runs the trunk $\mathcal{M}$ once per patch (see Fig. 2 for the full architecture), leaving most byte predictions to rely on a stale representation from the previous patch. *Right:* SP inserts transient scratchpads at selected byte positions, each aggregating the bytes seen so far within the patch and refreshing the trunk representation for subsequent predictions. This reduces patch lag while leaving the number of committed patch states unchanged. Other architectural components are omitted for clarity.

patch lag. In a standard autoregressive patch-based model, only the final byte within each patch can use the completed representation of that patch, while every earlier byte must rely on the previous patch-level context to preserve causality (Section 2.2). As patches grow larger, this lag widens and makes modeling quality increasingly sensitive to patch size.

In this work, we introduce *Scratchpad Patching* (SP), which decouples compute allocation from patch size to address patch lag (Section 3). Rather than committing a single representation only at each patch boundary, SP inserts transient *scratchpads* at selected internal byte positions (Fig. 1). Each scratchpad aggregates the bytes seen so far within the patch and serves subsequent byte predictions until the next scratchpad or the committed patch representation is produced (Section 3.1). Because within-patch scratchpads are excluded from the persistent KV cache at inference, they leave the committed patch sequence length and the resulting KV-cache footprint unchanged (Section 3.2). Among several strategies we evaluate, triggering scratchpads via next-byte prediction entropy is most effective, selectively allocating compute to information-dense regions; the same machinery also enables post-hoc adjustment of inference-time compute without retraining.

SP is a generic technique applicable to many existing patch-based architectures. Across experiments, SP improves the empirical frontier of quality versus patch size (Section 4.2): models can use larger patches and smaller KV caches without the usual quality penalty. With SP in place, different patching strategies from previous work cluster in performance-FLOPs space, indicating that the primary bottleneck may be insufficient compute rather than suboptimal boundary placement (Section 4.3). Further analyses show that under FLOPs-matched comparisons, SP matches or improves non-SP baselines on three of the four patchifier families, confirming that much of the gain comes from better-targeted rather than additional compute (Section 5.1). Our contributions are as follows.

- We introduce *Scratchpad Patching*, a general mechanism that decouples compute from patch size to reduce *patch lag*, which we characterize as a structural failure mode of patch-based models.
- We show that SP improves the empirical frontier of quality versus average patch size across downstream tasks; even at 16 bytes per patch, SP models can match or closely approach the byte-level baseline with a $16\times$ smaller KV cache over patches and 3–$4\times$ less inference compute.
- We find that with SP in place, the performance gap among patchifier families narrows substantially under comparable FLOPs budgets, and simple schemes such as fixed-size patching become competitive with complex boundary strategies.

2 Background

2.1 Tokenizer-based Language Modeling

Most modern language models operate on tokenized text [Bengio et al., 2003, Devlin et al., 2019, Brown et al., 2020, OpenAI, 2023, Google et al., 2023]. Given a raw text string, a tokenizer maps it to a discrete sequence of tokens $t_1, \ldots, t_M$, where each token typically corresponds to a subword

unit [Gage, 1994, Schuster and Nakajima, 2012, Wu et al., 2016, Sennrich et al., 2016, Kudo and Richardson, 2018, Kudo, 2018, Dagan et al., 2024, Liu et al., 2025]. The model is trained to maximize the log-likelihood $\sum_{i=1}^{M} \log p(t_i \mid t_{<i})$ of the observed token sequence.

Tokenization reduces the input sequence length and defines tokens as the *atomic* prediction units of the model. While effective, this external preprocessing step couples the model to a fixed segmentation scheme and can introduce brittleness (Section 1).

2.2 Patch-based Byte-level Modeling

These limitations have motivated tokenizer-free approaches that operate directly on bytes. In byte-level language modeling, the input becomes a UTF-8 byte sequence $b_1, \ldots, b_N$, where each $b_i \in \{0, \ldots, 255\}$.[2] The model defines an autoregressive distribution $p(b_i \mid b_{<i})$ to enable end-to-end modeling without tokenization. Because byte sequences are substantially longer than token sequences, a recent line of work explores *patch-based byte-level models*, which aggregate contiguous bytes into higher-level *patches* and reduce the number of sequence elements processed by the main trunk.

Architecture. Most patch-based architectures share a common design with five components (Fig. 2): an *encoder*, a *patchifier*, a *main trunk*, an *unpatchifier*, and a *decoder*. The encoder $\mathcal{E}$, main trunk $\mathcal{M}$, and decoder $\mathcal{D}$ are all stacks of causal Transformer layers, while the patchifier and unpatchifier mediate between byte-level and patch-level representations.

The encoder $\mathcal{E}$ maps the byte sequence to contextual representations $x = \mathcal{E}(b)$. The patchifier $\mathcal{P}$ partitions the byte sequence into L contiguous segments $[s_\ell, e_\ell]$ for each $\ell \in \{1, \ldots, L\}$ and produces patch-level representations $z_\ell := \text{Aggregate}\,(x_{s_\ell:e_\ell})$ via local cross-attention, using the mean-pooled segment embedding as the query. Together with a <bos> sentinel z_0, these form the patch sequence $z_0, z_1, \ldots, z_L$. The main trunk $\mathcal{M}$, which allocates the majority of model parameters and compute, processes the patch sequence as $\widetilde{z} = \mathcal{M}(z)$.

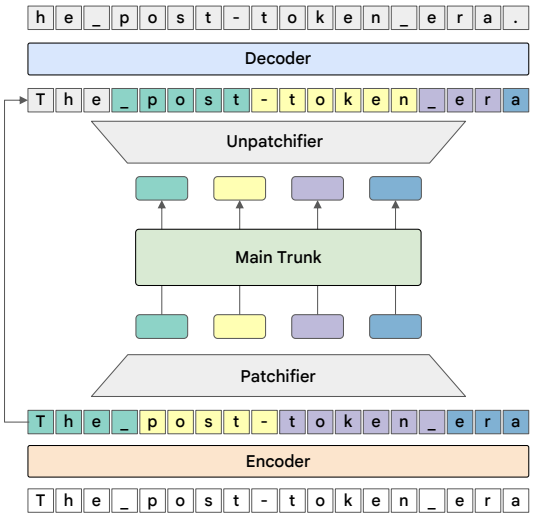


Figure 2: **Patch-based byte-level architecture.**

The unpatchifier $\mathcal{U}$ lifts patch-level trunk outputs back to byte positions and fuses them with encoder outputs x via a residual connection [Hwang et al., 2025]. Causality introduces an asymmetry: only the final byte of each patch ($n = e_\ell$) can condition on the *current* patch's trunk output, while all earlier bytes must instead rely on the output of the *previous* patch. We refer to the gap between a byte's prediction and the most recent patch-level representation available to it as *patch lag*. In our backbone, it takes the following form[3]

$$u_n = \begin{cases} \widetilde{z}_{\ell-1} + x_n, & \text{if } n \neq e_\ell, \\ \widetilde{z}_\ell + x_n, & \text{if } n = e_\ell. \end{cases} \tag{1}$$

Finally, the decoder $\mathcal{D}$ maps the resulting byte-level representations to next-byte prediction logits.

Patch Lag. Standard patch-based models treat each patch as an *atomic* unit in the trunk. Consequently, trunk compute is governed primarily by the number of patches L, regardless of how many bytes or how much internal structure each patch represents. This tightly couples the capacity to patch size: as the average bytes per patch grow, patch lag widens, where non-final byte positions condition on an increasingly stale patch-level representation, resulting in the trade-off between shorter sequences and modeling quality. Our approach, introduced next, directly addresses this limitation.

[2] In practice we expand the vocabulary beyond 256 to reserve IDs for sentinel tokens. In our experiments, the vocabulary size is set to 320 with the last 64 IDs reserved for sentinels such as <bos> and <pad>.

[3] We omit linear projections that match trunk and encoder outputs to the decoder dimension; see Section B for full details.

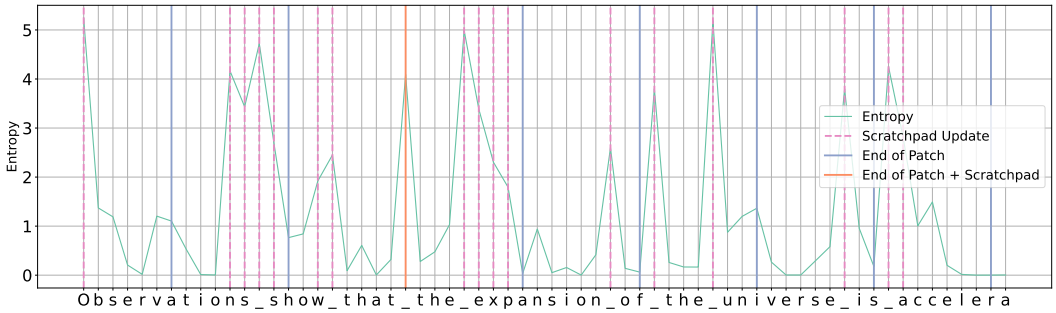


Figure 3: **Scratchpad Patching dynamics on fixed-size patching ($p = 8$).** Patch boundaries (**solid blue**) are regular by construction, while scratchpad updates (**dashed pink**) are triggered adaptively whenever the encoder's next-byte entropy (**green**) exceeds threshold $\tau_{\text{SP}} = 1.5$. When a scratchpad-trigger coincides with a patch boundary, patchification takes precedence (**solid orange**).

3 Scratchpad Patching

Scratchpad Patching (SP) reduces patch lag without altering the patch sequence, decoupling compute allocation from patch size. Instead of mapping each patch to a single representation, SP introduces a sequence of *scratchpad states* that progressively refine the patch representation by aggregating successively longer spans of bytes within the patch and passing each through the trunk. Because these states are used for computation but not persisted in the KV cache, each patch can undergo multiple internal refinement steps without increasing the inference-time KV-cache footprint. Fig. 1 provides intuition; we formalize scratchpad states and their interaction with patchification below.

3.1 Patchification with Scratchpads

For each patch ℓ spanning byte positions $[s_\ell, e_\ell]$, SP associates each position n with a binary indicator $p_n \in \{0, 1\}$ specifying whether a scratchpad update fires at n. SP is agnostic to the choice of patchifier; if a position is both a patch boundary and a scratchpad trigger, patchification takes precedence and the scratchpad update is suppressed. These indicators induce a sequence of scratchpad states $z_\ell^1, \ldots, z_\ell^{T_\ell}$ for patch ℓ, where $T_\ell := \sum_{j=s_\ell}^{e_\ell} p_j$ counts the total updates and $T_\ell = 0$ recovers the standard patch-based model. T_ℓ may vary across patches, allowing the model to adaptively allocate more compute to longer or more information-dense patches. We reserve z_ℓ for the *committed* patch representation $z_\ell := \text{Aggregate}(x_{s_\ell:e_\ell})$, and z_ℓ^t for the *transient* t-th scratchpad.

For any position $s_\ell \le n \le e_\ell$, let $t = \sum_{j=s_\ell}^{n} p_j$ index the scratchpad fired so far in patch ℓ. When $p_n = 1$, we form $z_\ell^t = \text{Aggregate}(x_{s_\ell:n})$ over this prefix and pass it through $\mathcal{M}$ identically as a regular patch state, yielding $\widetilde{z}_\ell^t$, which is then broadcast to byte positions for the decoder $\mathcal{D}$. Adopting the convention $\widetilde{z}_\ell^0 := \widetilde{z}_{\ell-1}$ before any scratchpad fires within the current patch, Eq. (1) becomes

$$u_n = \begin{cases} \widetilde{z}_\ell^t + x_n, & \text{if } n \neq e_\ell, \\ \widetilde{z}_\ell + x_n, & \text{if } n = e_\ell. \end{cases} \tag{2}$$

The essence of SP is replacing $\widetilde{z}_{\ell-1}$ in Eq. (1) with $\widetilde{z}_\ell^t$: each non-final byte now conditions on a fresh scratchpad state from the *current* patch, rather than the stale representation from the previous patch. Patch lag is thus reduced from one full patch to the gap to the most recent scratchpad.

Selective Scratchpad Updating. A simple instantiation of SP applies a scratchpad update at every byte position. This minimizes patch lag but incurs compute comparable to a vanilla byte-level model, negating the efficiency benefits of patchification. Empirically, such dense updates also yield diminishing returns over selective updating (Section E.2). For adaptive, content-aware compute allocation, we instead parameterize the trigger using next-byte prediction entropy, derived from a language modeling (LM) head applied to the encoder outputs x. Specifically, a scratchpad update is issued whenever the encoder's prediction entropy $H_n := -\sum_{b \in \mathcal{V}} p(b \mid x_{\le n}) \log p(b \mid x_{\le n})$ exceeds a predefined threshold: $p_n := \mathbf{1}_{[H_n > \tau_{\text{SP}}]}$. Fig. 3 illustrates this on a sample sequence with fixed-size patching: scratchpad updates fire at positions of elevated next-byte entropy, while patch

boundaries remain on a regular fixed-size grid. We ablate updating strategies in Section E.2 and provide additional qualitative case studies in Section E.3.

3.2 Implementation

Parallel Training with Specialized Attention Masking. During training, scratchpad states are unrolled and concatenated into the trunk's input sequence so that the loss can be computed over all byte positions in parallel,

$$\mathbf{z} = \big[z_0, \underbrace{z_1^1, \ldots, z_1^{T_1}, z_1}_{\text{patch } 1}, \underbrace{z_2^1, \ldots, z_2^{T_2}, z_2}_{\text{patch } 2}, \ldots, \underbrace{z_L^1, \ldots, z_L^{T_L}, z_L}_{\text{patch } L}\big],$$

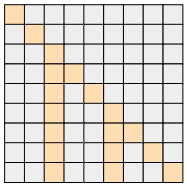


Figure 4: **Attention mask for SP training.**

where z_0 is a <bos> sentinel and each patch contributes T_ℓ scratchpads followed by its committed representation z_ℓ; patches with $T_\ell = 0$ collapse to $[\ldots, z_\ell]$, recovering the standard patch-based layout. Self-attention in $\mathcal{M}$ is governed by a specialized causal mask (Fig. 4): every scratchpad or committed element of patch ℓ attends only to (i) itself and (ii) committed representations $z_{\ell'}$ from earlier patches $\ell' < \ell$. All scratchpads associated with patch ℓ share the same position index as the committed patch state. Crucially, scratchpads are never attended to by other elements, so refinement arises not from within-trunk recurrence but from the growing partial-aggregation span. This design allows all scratchpads to be processed in parallel during training and licenses their removal from the KV cache at inference.

While SP increases training-time compute by introducing transient states, the total FLOPs are comparable to training a non-SP model with a smaller patch size, e.g., one where all scratchpad positions act as patch boundaries, though the attention patterns differ due to the specialized mask.

Efficient Inference with Scratchpad Overriding. At inference time, scratchpad states are transient: only each patch's finalized representation is retained in the KV cache of the trunk $\mathcal{M}$ and exposed to subsequent patches, while scratchpads are computed on the fly and immediately overridden, incurring no additional KV-cache overhead.

4 Experiments

We empirically evaluate Scratchpad Patching (SP), focusing on the trade-offs among quality, persistent sequence length, and compute. We describe the experimental setup in Section 4.1, present the main results in Section 4.2, and analyze the role of compute allocation in Section 4.3.

4.1 Setup

Models. All patch-based byte-level models in our experiments share the same encoder-trunk-decoder backbone (Section 2) and differ primarily in their patchification mechanism. We refer to each variant by its patchification strategy; labels denote the patchifier re-implemented within our shared backbone, not exact reproductions of the original model architectures, which may differ in other design choices and hyperparameters. We study four patchifier families: (i) **Fixed-size patching** [Clark et al., 2022, Nawrot et al., 2022, Yu et al., 2023], which groups bytes into non-overlapping windows of fixed width $p \in \{2, 4, 8, 16\}$; (ii) **SpaceByte patching** [Slagle, 2024], which places patch boundaries at whitespace-like delimiters, producing variable-length patches; (iii) **Entropy-based patching** [Nawrot et al., 2023, Pagnoni et al., 2024], where an auxiliary LM head on top of the encoder computes next-byte prediction entropy and marks positions above a threshold as patch boundaries; and (iv) **H-Net patching** [Hwang et al., 2025], which uses a learned router to score each byte position and determine boundaries. For each baseline, we train and evaluate its SP variant with entropy-based scratchpad updates. We also include standard **byte-level** and **tokenizer-based** baselines. All models have $\sim$2B parameters; full architectural details are in Section A.

Training. All models are pretrained on the same mixture of open-source datasets spanning code, natural language, and mathematics (Section C.1) under a *fixed-data* regime of $\sim$400B raw bytes. Total training FLOPs therefore differ across models, owing to their distinct average bytes per patch (or token) and scratchpad allocations. Optimization hyperparameters are detailed in Section C.2.

Evaluation. We evaluate (i) Bits-Per-Byte (BPB) on held-out validation data, (ii) estimated pass@1 on code generation with MBPP [Austin et al., 2021] and HumanEval [Chen et al., 2021], and (iii) accuracy on multiple-choice natural language understanding benchmarks. As efficiency proxies, we report the persistent *sequence reduction factor*, the average number of input bytes mapped to one sequence element (a byte, token, or committed patch, depending on the model), and FLOPs/byte reduction, both measured relative to the byte-level baseline. Full evaluation details are in Section D.

4.2 Main Results

Improved Quality-Efficiency Trade-off. Fig. 5 plots validation BPB against the sequence reduction factor. Across all patchifier families, SP consistently shifts the frontier: at a fixed sequence-reduction target it achieves lower BPB, and at a fixed BPB target it supports larger patches. The dashed regression lines confirm a clear downward shift from baselines (dashed red) to their SP variants (dashed blue). The gains are most pronounced in aggressive patch-size regimes (e.g., $p = 8$ and $p = 16$), where vanilla models under-allocate compute to information-dense regions and suffer substantial BPB degradation. SP recovers much of this lost capacity through within-patch scratchpads, without changing the committed patch sequence length. We observe the same trend on downstream tasks (Section E.1).

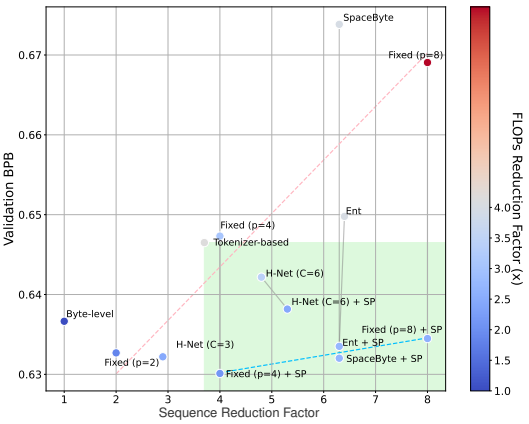


Figure 5: Validation BPB versus sequence reduction factor. Points are colored by training FLOPs reduction factor (blue: higher than tokenizer; gray: comparable; red: lower). Dashed regression lines summarize trends for non-SP baselines (red) and their SP counterparts (blue). The green-shaded region marks variants that use shorter sequences than the tokenizer baseline and have lower BPB.

The coloring in Fig. 5 further reveals that SP models achieve substantially better BPB than the byte-level baseline while retaining short trunk sequences and FLOPs savings. Compared to the tokenizer baseline, SP models can be both lower in BPB and run on shorter trunk sequences (green-shaded region), albeit with moderately higher training FLOPs.

Natural Language Understanding. Table 1 reports downstream accuracy on eight multiple-choice NLU benchmarks. We report sequence length and FLOPs/byte reduction measured during validation BPB evaluation as efficiency proxies. Within each patchifier family, SP improves average task accuracy and largely recovers the degradation incurred by aggressive patch sizes: Fixed (p=16) improves from 48.0 to 54.2 with SP, matching the byte-level baseline (54.1) despite operating at 16 bytes per patch. After adding SP, simple schemes (e.g., fixed-size patching and SpaceByte) match or surpass more sophisticated strategies, and the gap among patchifier families narrows substantially.

Most SP variants outperform the byte-level baseline while running on a shorter patch sequence, suggesting that patching can provide a useful abstraction that lets the model concentrate compute on higher-level structure rather than redundant byte-level detail. The tokenizer-based model is a strong baseline on downstream NLU tasks, outperforming both the byte-level model and most non-SP patch-based models. We attribute this to the strong inductive bias of subword tokenization for language. Several SP variants match or surpass the tokenizer at shorter trunk sequences and without relying on language-specific biases, despite higher training FLOPs.

Code Generation. We next evaluate whether SP improves downstream *generation* quality. Table 2 reports pass@1 rates on MBPP and HumanEval alongside inference-time KV-cache and FLOPs/byte reduction. Across patchifier families, SP consistently improves pass@1 while largely preserving the KV-cache reduction factor. At larger patch sizes, these gains come with FLOPs reduction comparable to or larger than the tokenizer baseline. Simple schemes, such as fixed-size patching and SpaceByte, are already strong baselines for code, and SP extends this advantage to large-patch regimes (p=8, p=16), recovering most of the quality lost. In contrast to the NLU setting, the tokenizer-based model is a weak baseline for code generation in our setup, underperforming both the byte-level and various

Table 1: Downstream task accuracy (↑) on natural language understanding (NLU) benchmarks. **Bold** and underlined entries indicate the best and second-best results per task, respectively. Sequence and FLOPs/byte reduction are measured during evaluation on validation BPB. †SP influences the training dynamics of learned patchifiers, resulting in slightly different factors compared to the baselines.

Model	Sequence Reduction (↑)	FLOPs/byte Reduction (↑)	ARC-E	ARC-C	BoolQ	HSwag	OBQA	PIQA	WinoG	MMLU	Avg.
Byte-level	1.0	1.0	66.6	38.7	64.5	55.1	45.2	72.8	56.0	33.5	54.1
Tokenizer-based	3.7	4.1	**71.6**	40.7	60.7	58.4	**49.2**	73.6	57.8	**35.1**	55.9
Fixed ($p = 4$)	4.0	3.1	67.4	39.8	50.5	57.9	44.8	70.4	58.6	33.8	52.9
Fixed ($p = 4$) + **SP**	4.0	2.1	71.0	41.6	65.2	**59.7**	46.0	72.9	58.8	34.7	**56.2**
Fixed ($p = 8$)	8.0	4.5	64.9	39.6	57.2	54.5	41.6	71.0	56.2	32.6	52.2
Fixed ($p = 8$) + **SP**	8.0	2.6	68.5	39.8	53.3	59.2	44.4	73.0	57.4	34.6	53.8
Fixed ($p = 16$)	16.0	5.7	56.5	35.2	54.3	47.3	38.2	67.7	55.2	29.8	48.0
Fixed ($p = 16$) + **SP**	16.0	2.9	67.5	40.8	60.0	58.0	43.4	72.5	58.2	33.5	54.2
SpaceByte	6.3	4.0	67.3	38.6	63.3	57.3	44.8	72.8	57.7	33.9	54.5
SpaceByte + **SP**	6.3	2.7	69.8	**41.9**	64.2	59.5	48.0	**73.8**	**59.0**	33.7	**56.2**
Entropy-based	6.4	4.0	66.1	40.6	50.6	57.7	46.4	72.4	57.9	33.7	53.2
Entropy-based + **SP**	6.3†	2.7	66.0	40.4	**66.3**	**59.7**	45.4	72.8	57.4	34.4	55.3
H-Net	4.8	3.4	70.3	40.1	64.5	58.5	45.6	73.7	56.7	33.9	55.4
H-Net + **SP**	5.3†	2.4	67.0	40.9	65.9	59.6	45.8	72.1	**59.0**	33.6	55.5

patch-based models. SP-augmented models widen this gap further, while offering larger KV-cache reductions over the tokenizer and preserving inference FLOPs efficiency. These results suggest that SP offers a better quality-efficiency trade-off for code generation tasks.

4.3 Compute Allocation Narrows the Gap Among Patchifier Choices

The results above suggest that quality differences across patchifiers are driven less by the exact boundary rule and more by how compute is distributed across patches. To make this explicit, Fig. 6 plots validation BPB against *training-time* FLOPs reduction. Standard patchifiers save FLOPs by shortening the trunk sequence, but at the cost of higher BPB as patch size grows. SP moves models to a better region of this trade-off: it injects additional compute via selective within-patch refinements, and does so in a way that yields disproportionately large BPB gains. After adding SP, multiple patchifier families cluster tightly in the BPB-FLOPs space, suggesting that compute allocation may matter more than the choice of patchification. We hypothesize that this is partly because attention layers within the trunk blur explicit patch boundaries, and SP compensates for imperfect or misaligned patchification by providing additional refinement opportunities.

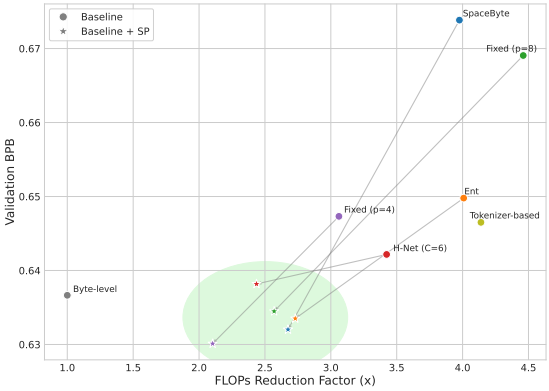


Figure 6: Validation BPB versus training FLOPs reduction relative to the byte-level baseline.

5 Analyses

We analyze SP along several dimensions: FLOPs-matched comparisons (Section 5.1), multilingual performance (Section 5.2), and inference-time flexibility (Section 5.3). Additional results, including ablations of scratchpad triggering strategies and qualitative case studies, are provided in Section E.

5.1 FLOPs-matched Performance Comparison

We explore whether the gains from SP simply reflect additional compute by comparing SP models against their non-SP counterparts under the same total training FLOPs. As shown in Fig. 7, for most patchifier families, SP matches or improves BPB at equal training compute. This confirms that the effectiveness of SP arises from better-targeted compute allocation rather than a larger compute budget. The main exception is H-Net, where SP can degrade validation BPB under strict FLOPs matching.

Table 2: Performance and efficiency comparison on MBPP and HumanEval. Inference KV-cache and FLOPs/byte reduction are relative to the byte-level baseline. †SP influences the training dynamics of learned patchifiers, resulting in slightly different patch sizes compared to the respective baselines.

Model	MBPP			HumanEval		
	Pass@1 (% ↑)	**KV Cache Reduction** (↑)	**Inf. FLOPs/byte Reduction** (↑)	**Pass@1** (% ↑)	**KV Cache Reduction** (↑)	**Inf. FLOPs/byte Reduction** (↑)
Byte-level	26.3	1.0×	1.0×	15.5	1.0×	1.0×
Tokenizer-based	23.3	2.6×	3.1×	13.3	3.0×	3.5×
Fixed (p=4)	28.1	4.0×	3.1×	16.8	4.0×	3.1×
Fixed (p=4) + **SP**	31.9	4.0×	2.5×	17.3	4.0×	2.7×
Fixed (p=8)	24.1	8.0×	4.4×	13.0	8.0×	4.4×
Fixed (p=8) + **SP**	32.1	8.0×	3.2×	15.9	8.0×	3.7×
Fixed (p=16)	18.2	16.0×	5.6×	10.5	16.0×	5.6×
Fixed (p=16) + **SP**	27.5	16.0×	3.7×	14.8	16.0×	4.5×
SpaceByte	26.5	5.4×	3.7×	15.7	6.3×	4.0×
SpaceByte + **SP**	30.3	5.4×	2.9×	16.6	6.3×	3.5×
Entropy-based	23.2	9.1×	4.7×	12.6	11.8×	5.1×
Entropy-based + **SP**	27.2	8.8×†	3.3×	16.3	11.7×†	4.2×
H-Net	27.6	4.7×	3.4×	15.1	5.8×	3.8×
H-Net + **SP**	28.5	4.7×	3.0×	15.4	6.1×†	3.6×

We hypothesize that this stems from inefficient interactions between scratchpad updates and learned patch boundaries, leading to redundant compute (see the qualitative analysis in Section E.3).

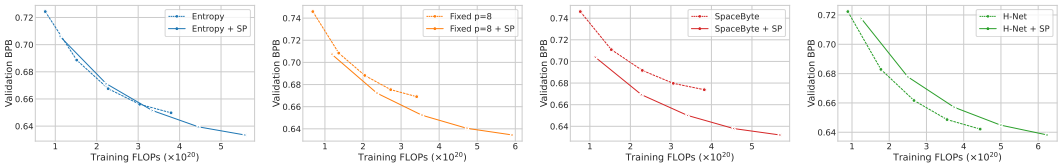


Figure 7: **Validation BPB performance comparison under matched training FLOPs**, broken down by different patchifier families.

5.2 Multilingual Performance

To assess robustness across languages, we evaluate all models on the FLORES-200 validation set [Costa-Jussà et al., 2022]. Fig. 8 shows the average BPB rank of each model across 200 languages (lower is better). The pure byte-level model achieves the strongest and most consistent performance, while the tokenizer-based model performs poorly on average, reflecting biases tied to specific scripts and morphologies. Adding SP consistently improves the ranking of patch-based byte-level models, narrowing the gap to the byte-level baseline across languages.

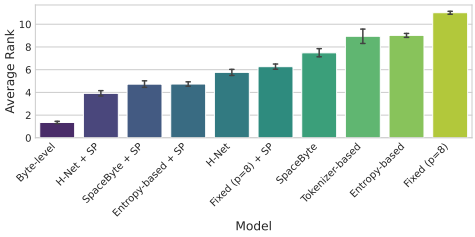
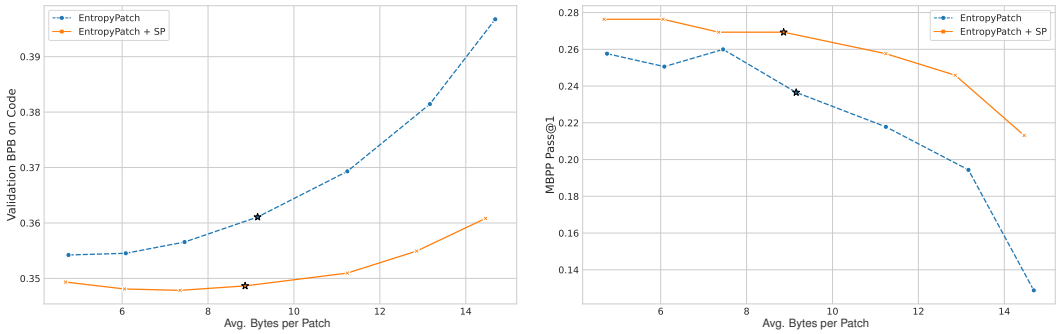


Figure 8: Average BPB rank across 200 languages of the FLORES-200 validation set. Error bars represent the standard error of the mean rank across languages.

5.3 Inference-time Compute Adjustment

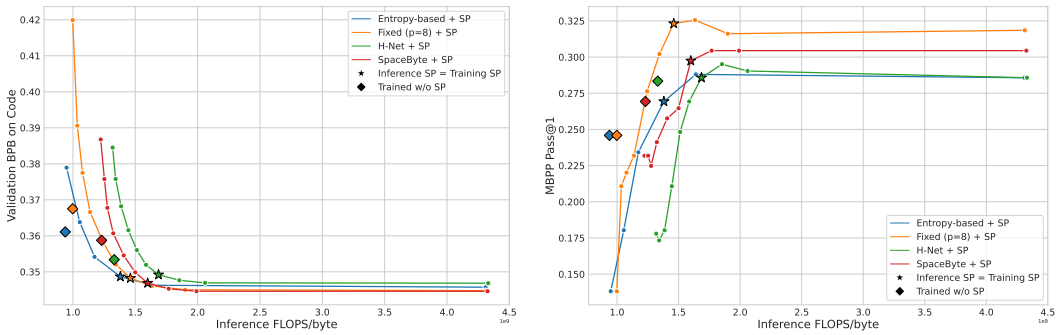
A key practical advantage of SP is that inference-time compute can be adjusted *post-hoc* without retraining. We evaluate this flexibility along two axes at inference time: (i) varying the patch size and (ii) varying the scratchpad update frequency.

Patch Size Variation. We use entropy-based patching with threshold $\tau_P = 2.5$ as the default model and vary τ_P at inference time to induce different realized average patch sizes. As shown in Figs. 9a and 9b, models trained with SP exhibit strong robustness to inference-time changes in patch size:



(a) Validation BPB on code. (b) MBPP pass@1.

Figure 9: **Inference-time patch size variation.** Performance under different realized average patch sizes applied at inference time without retraining.



(a) Validation BPB on code. (b) MBPP pass@1.

Figure 10: **Inference-time adjustment of scratchpad frequency.** Performance as a function of inference FLOPs/byte when varying the scratchpad update frequency post-hoc without retraining.

performance degrades gracefully when the realized patch size deviates from the training configuration. In contrast, non-SP models suffer substantial performance drops under the same mismatch, suggesting that SP can compensate for suboptimal patch boundaries to a considerable degree.

Scratchpad Frequency Variation. Figs. 10a and 10b show that varying the scratchpad frequency at inference time yields a smooth trade-off curve in both validation BPB on code and MBPP pass@1. Reducing scratchpad density relative to the training default trades compute for modest quality loss, while increasing it recovers performance. The entire trade-off is exposed at inference without any retraining, giving SP a single-knob compute control that standard patch-based baselines do not have.

6 Related Work

Language Modeling Beyond Tokenization. The limitations of tokenizers have motivated work on alternative text representations, including morphology-driven byte encodings [Limisiewicz et al., 2024], gzip-based and neural model-based compressors [Jiang et al., 2023, Lester et al., 2024, Zheng et al., 2026], concept-level semantic units [Barrault et al., 2024], and pixel-rendered representations [Salesky et al., 2021, Rust et al., 2023, Lotz et al., 2023, Gao et al., 2024].

A separate line of recent research pursues byte-level modeling without external tokenization. ByT5 [Xue et al., 2022] adopts T5 encoder-decoder architectures [Raffel et al., 2020] at the byte level; MrT5 [Kallini et al., 2024] extends ByT5 with a token deletion gating mechanism to dynamically reduce the sequence length while preserving model performance; MambaByte [Wang et al., 2024] leverages efficient Mamba [Gu and Dao, 2024, Dao and Gu, 2024, Lahoti et al., 2026] layers to handle long byte sequences; and EvaByte [Zheng et al., 2025] combines multi-token prediction [Stern et al., 2018, Gloeckle et al., 2024, Cai et al., 2024, Grivas et al., 2025] with improved linear attention

[Zheng et al., 2022, 2023] to improve byte-level efficiency. To address the computational cost of long byte sequences, *patch-based* architectures have been explored, which we discuss next.

Patch-based Architectures for Byte Modeling. Patch-based byte-level models pool contiguous bytes into shorter patch-level or latent representations that are processed by a main trunk (Section 2). Existing designs differ primarily in how patch boundaries are determined. *Static* methods group bytes into fixed-size windows, as in Funnel-Transformer [Dai et al., 2020], Canine [Clark et al., 2022], Hourglass transformers [Nawrot et al., 2022], MegaByte [Yu et al., 2023], and Block Transformers [Ho et al., 2024], or use cross-attention to compress raw inputs to a fixed number of latent vectors, as in the Perceiver family [Jaegle et al., 2021b,a, Hawthorne et al., 2022]. *Dynamic* approaches adapt segmentation to the input content. Dynamic Pooling Transformers [DPT; Nawrot et al., 2023] explore multiple dynamic boundary strategies spanning delimiter-based, entropy-based, and learned schemes. Delimiter-based methods place boundaries at whitespace-like positions, as also explored in SpaceByte [Slagle, 2024] and AU-Nets [Videau et al., 2025]. Entropy-based methods use local uncertainty to guide segmentation; Byte Latent Transformers [BLT; Pagnoni et al., 2024] leverage entropies pre-computed by a separate model offline. In contrast, the entropy-based baseline in our work applies a language modeling head on top of the encoder $\mathcal{E}$ to compute entropies online, avoiding the offline-model dependency of BLTs. Alternatively, the boundary predictor can be parameterized and learned explicitly. Charformer [Tay et al., 2022] uses gradient-based tokenization to form latent subwords by pooling the byte sequence at multiple resolutions; MANTa [Godey et al., 2022] predicts boundaries by modeling the byte-block assignment; and H-Nets [Hwang et al., 2025] leverage a simple yet effective design with ratio loss to enable stable boundary training, compatible with multi-stage hierarchical patching. Bolmo [Minixhofer et al., 2025] presents an effective framework byteifying existing tokenizer-based models via non-causal boundary prediction aligned with tokenization behaviors. Dynamic chunking has also been applied in the image domain for spatially adaptive segmentation [Haridas et al., 2026], and in DLCM [Qu et al., 2025] and ConceptMoE [Huang et al., 2026] for compressing tokens into higher-level concept units.

Orthogonal to work on improving patchification schemes, our approach enables fine-grained, content-adaptive compute allocation that is independent of both patch size and the boundary rule, yielding a substantially better trade-off among persistent sequence length, compute, and task performance.

Adaptive Compute Allocation. The central idea behind SP is decoupling compute allocation from patch size or persistent sequence length, connecting to a broader line of work on adaptive computation [Bengio et al., 2015, Huang et al., 2016, Graves, 2016]. Universal transformers or looped language models [Dehghani et al., 2019, Csordás et al., 2024, Tan et al., 2023, Giannou et al., 2023, Zhu et al., 2025, Wu et al., 2025a, Bae et al., 2025a, Geiping et al., 2025, Bae et al., 2025b, Zeng et al., 2026] recurrently apply shared parameters to achieve dynamic computation, and PonderNet [Banino et al., 2021] learns to control the number of recurrent steps end-to-end. Pause tokens [Goyal et al., 2024] allow models to exploit additional inference-time computation. Mixture-of-Depths [Raposo et al., 2024] introduces per-layer routers to selectively pass a subset of tokens for full computation while the remainder bypass the layer through the residual connection. The PHD-Transformer [Wu et al., 2025b] repeats input tokens for pretraining length scaling, retaining only the KV cache of the original tokens via a specialized attention mask. While motivated by length scaling, this shares with SP the principle of introducing transient states that participate in computation but are excluded from the persistent KV cache at prediction time. SP can be viewed as applying adaptive compute at the patch level of byte-level architectures. SP triggers scratchpads within each patch based on next-byte prediction entropy, selectively allocating compute to information-dense regions of the byte stream, without increasing the inference-time KV-cache footprint.

7 Conclusion

In this work, we introduced *Scratchpad Patching (SP)*, a general mechanism for improving patch-based tokenizer-free byte-level models. By inserting transient scratchpad states within patches, SP decouples compute allocation from the effective patch size and addresses *patch lag*, the structural gap between next-byte predictions and the most recent patch representation available. Empirically, SP consistently shifts the quality-efficiency frontier across a wide range of patching schemes. Our results highlight SP as a practical step toward more flexible end-to-end language models that decide where to allocate compute, rather than inheriting that decision from a tokenizer or patchifier.

Limitations. This work primarily focuses on patch-based byte-level models that apply a single stage of patching to the input sequence. While SP is in principle compatible with hierarchical architectures that employ multi-stage patching, such as in H-Net [Hwang et al., 2025] or AU-Nets [Videau et al., 2025], we leave a systematic study of these settings to future work. In addition, we explore a simple form of scratchpads that recompute intermediate representations when new bytes become available; richer update rules that draw connections to recurrent methods may further improve efficiency and expressivity. Finally, SP does not directly reduce training-time FLOPs compared to standard patch-based baselines, and designing scratchpad formulations that yield more explicit compute savings remains an important direction for future research.

References

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Valentin Hoffman, Tomasz Limisiewicz, Yulia Tsvetkov, and Noah A Smith. Magnet: Improving the multilingual fairness of language models with adaptive gradient-based tokenization. *arXiv preprint arXiv:2407.08818*, 2024.

Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. Character-level language modeling with deeper self-attention. In *Proceedings of the AAAI conference on artificial intelligence*, 2019.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise loRA. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL https://openreview.net/forum?id=WwpYSOkkCt.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b. URL https://openreview.net/forum?id=QuqsEIVWIG.

Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. In *8th ICML Workshop on Automated Machine Learning (AutoML)*, 2021. URL https://openreview.net/forum?id=1EuxRTe0WN.

Loïc Barrault, Paul-Ambroise Duquenne, Maha Elbayad, Artyom Kozhevnikov, Belen Alastruey, Pierre Andrews, Mariano Coria, Guillaume Couairon, Marta R Costa-jussà, David Dale, et al. Large concept models: Language modeling in a sentence representation space. *arXiv preprint arXiv:2412.08821*, 2024.

Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael K Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xLSTM: Extended long short-term memory. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=ARAxPPIAhq.

Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. Conditional computation in neural networks for faster models. *arXiv preprint arXiv:1511.06297*, 2015.

Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of machine learning research*, 2003.

Yonatan Bisk, Rowan Zellers, Ronan Le bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020. URL https://ojs.aaai.org/index.php/AAAI/article/view/6239.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020. URL https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Forty-first International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=PEpbUobfJv.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Dokook Choe, Rami Al-Rfou, Mandy Guo, Heeyoung Lee, and Noah Constant. Bridging the gap for tokenizer-free language models. *arXiv preprint arXiv:1908.10322*, 2019.

Junyoung Chung, Sungjin Ahn, and Yoshua Bengio. Hierarchical multiscale recurrent neural networks. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=S1di0sfgl.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019. URL https://aclanthology.org/N19-1300.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 2022. URL https://aclanthology.org/2022.tacl-1.5.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Marta R Costa-Jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, et al. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*, 2022.

Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D Manning. MoEUT: Mixture-of-experts universal transformers. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=ZxVrkm7Bjl.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Roziere. Getting the most out of your tokenizer for pre-training and domain adaptation. *arXiv preprint arXiv:2402.01035*, 2024.

Zihang Dai, Guokun Lai, Yiming Yang, and Quoc Le. Funnel-transformer: Filtering out sequential redundancy for efficient language processing. *Advances in neural information processing systems*, 33:4271–4282, 2020.

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://proceedings.mlr.press/v235/dao24a.html.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=HyzdRiR9Y7.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL https://aclanthology.org/N19-1423.

Philip Gage. A new algorithm for data compression. *C Users Journal*, 1994.

Tianyu Gao, Zirui Wang, Adithya Bhaskar, and Danqi Chen. Improving language understanding from screenshots. *arXiv preprint arXiv:2402.14073*, 2024.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.

Angeliki Giannou, Shashank Rajput, Jy-Yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL https://proceedings.mlr.press/v202/giannou23a.html.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.

Nathan Godey, Roman Castagné, Éric de la Clergerie, and Benoît Sagot. MANTa: Efficient gradient-based tokenization for end-to-end robust language modeling. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2022. URL https://aclanthology.org/2022.findings-emnlp.207.

Gemini Team Google, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=ph04CRkPdC.

Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Andreas Grivas, Lorenzo Loconte, Emile van Krieken, Piotr Nawrot, Yu Zhao, Euan Wielewski, Pasquale Minervini, Edoardo Ponti, and Antonio Vergari. Fast and expressive multi-token prediction with probabilistic circuits, 2025.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=tEYskw1VY2.

Yuling Gu, Oyvind Tafjord, Bailey Kuehl, Dany Haddad, Jesse Dodge, and Hannaneh Hajishirzi. OLMES: A standard for language model evaluations. *arXiv preprint arXiv:2406.08446*, 2024.

Akash Haridas, Utkarsh Saxena, Parsa Ashrafi Fashi, Mehdi Rezagholizadeh, Vikram Appia, and Emad Barsoum. Dynamic chunking diffusion transformer. *arXiv preprint arXiv:2603.06351*, 2026.

Curtis Hawthorne, Andrew Jaegle, Cătălina Cangea, Sebastian Borgeaud, Charlie Nash, Mateusz Malinowski, Sander Dieleman, Oriol Vinyals, Matthew Botvinick, Ian Simon, Hannah Sheahan, Neil Zeghidour, Jean-Baptiste Alayrac, Joao Carreira, and Jesse Engel. General-purpose, long-context autoregressive modeling with perceiver AR. In *Proceedings of the 39th International Conference on Machine Learning*, pages 8535–8558, 2022. URL https://proceedings.mlr.press/v162/hawthorne22a.html.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

Namgyu Ho, Sangmin Bae, Taehyeon Kim, hyunjik.jo, Yireun Kim, Tal Schuster, Adam Fisch, James Thorne, and Se-Young Yun. Block transformer: Global-to-local language modeling for fast inference. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=6osgTNnAZQ.

Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *European conference on computer vision*, 2016.

Zihao Huang, Jundong Zhou, Xingwei Qu, Qiyang Min, and Ge Zhang. Conceptmoe: Adaptive token-to-concept compression for implicit compute allocation. *arXiv preprint arXiv:2601.21420*, 2026.

Kyuyeon Hwang and Wonyong Sung. Character-level language modeling with hierarchical recurrent neural networks. In *2017 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 5720–5724. IEEE, 2017.

Sukjun Hwang, Brandon Wang, and Albert Gu. Dynamic chunking for end-to-end hierarchical sequence modeling. *arXiv preprint arXiv:2507.07955*, 2025.

Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, et al. Perceiver io: A general architecture for structured inputs & outputs. *arXiv preprint arXiv:2107.14795*, 2021a.

Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. Perceiver: General perception with iterative attention. In *Proceedings of the 38th International Conference on Machine Learning*, 2021b. URL https://proceedings.mlr.press/v139/jaegle21a.html.

Zhiying Jiang, Matthew Yang, Mikhail Tsirlin, Raphael Tang, Yiqin Dai, and Jimmy Lin. "low-resource" text classification: A parameter-free classification method with compressors. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023. URL https://aclanthology.org/2023.findings-acl.426.

Julie Kallini, Shikhar Murty, Christopher D Manning, Christopher Potts, and Róbert Csordás. Mrt5: Dynamic token merging for efficient byte-level language models. *arXiv preprint arXiv:2410.20771*, 2024.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2018. URL https://aclanthology.org/P18-1007/.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018. URL https://aclanthology.org/D18-2012/.

Aakash Lahoti, Kevin Li, Berlin Chen, Caitlin Wang, Aviv Bick, J Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved sequence modeling using state space principles. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=HwCvaJOiCj.

Sander Land and Max Bartolo. Fishing for magikarp: Automatically detecting under-trained tokens in large language models. *arXiv preprint arXiv:2405.05417*, 2024.

Brian Lester, Jaehoon Lee, Alex Alemi, Jeffrey Pennington, Adam Roberts, Jascha Sohl-Dickstein, and Noah Constant. Training llms over neurally compressed text. *arXiv preprint arXiv:2404.03626*, 2024.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, et al. Datacomp-lm: In search of the next generation of training sets for language models. *arXiv preprint arXiv:2406.11794*, 2024.

Tomasz Limisiewicz, Terra Blevins, Hila Gonen, Orevaoghene Ahia, and Luke Zettlemoyer. Myte: Morphology-driven byte encoding for better and fairer multilingual language modeling. *arXiv preprint arXiv:2403.10691*, 2024.

Alisa Liu, Jonathan Hayase, Valentin Hofmann, Sewoong Oh, Noah A. Smith, and Yejin Choi. SuperBPE: Space travel for language models. In *Second Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=lcDRvffeNP.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Jonas Lotz, Elizabeth Salesky, Phillip Rust, and Desmond Elliott. Text rendering strategies for pixel language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://aclanthology.org/2023.emnlp-main.628.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. Starcoder 2 and the stack v2: The next generation, 2024.

Scott Lundberg and Marco Tulio Ribeiro. The art of prompt design: Prompt boundaries and token healing. *Medium*, 2023. URL https://towardsdatascience.com/the-art-of-prompt-design-prompt-boundaries-and-token-healing-3b2448b0be38.

Microsoft. Guidance, 2023. URL https://github.com/microsoft/guidance.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018. URL https://aclanthology.org/D18-1260.

Benjamin Minixhofer, Tyler Murray, Tomasz Limisiewicz, Anna Korhonen, Luke Zettlemoyer, Noah A Smith, Edoardo M Ponti, Luca Soldaini, and Valentin Hofmann. Bolmo: Byteifying the next generation of language models. *arXiv preprint arXiv:2512.15586*, 2025.

Piotr Nawrot, Szymon Tworkowski, Michał Tyrolski, Lukasz Kaiser, Yuhuai Wu, Christian Szegedy, and Henryk Michalewski. Hierarchical transformers are more efficient language models. In *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022. URL https://aclanthology.org/2022.findings-naacl.117.

Piotr Nawrot, Jan Chorowski, Adrian Lancucki, and Edoardo Maria Ponti. Efficient transformers with dynamic token pooling. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023. URL https://aclanthology.org/2023.acl-long.353.

Pit Neitemeier, Björn Deiseroth, Constantin Eichenberg, and Lukas Balles. Hierarchical autoregressive transformers: Combining byte-˜ and word-level processing for robust, adaptable language models. *arXiv preprint arXiv:2501.10322*, 2025.

OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Abraham Toluwase Owodunni, Orevaoghene Ahia, and Sachin Kumar. Flexitokens: Flexible tokenization for evolving language models. *arXiv preprint arXiv:2507.12720*, 2025.

Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, et al. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.

Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. Openwebmath: An open dataset of high-quality mathematical web text. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=jKHmjlpViu.

Xingwei Qu, Shaowen Wang, Zihao Huang, Kai Hua, Fan Yin, Rui-Jie Zhu, Jundong Zhou, Qiyang Min, Zihao Wang, Yizhi Li, et al. Dynamic large concept models: Latent reasoning in an adaptive semantic space. *arXiv preprint arXiv:2512.24617*, 2025.

Alec Radford, Rafal Jozefowicz, and Ilya Sutskever. Learning to generate reviews and discovering sentiment. *arXiv preprint arXiv:1704.01444*, 2017.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

Jessica Rumbelow and Matthew Watkins. Solidgoldmagikarp (plus, prompt generation), 2023. URL https://www.alignmentforum.org/posts/aPeJE8bSo6rAFoLqg/solidgoldmagikarp-plus-prompt-generation.

Phillip Rust, Jonas F. Lotz, Emanuele Bugliarello, Elizabeth Salesky, Miryam de Lhoneux, and Desmond Elliott. Language modelling with pixels. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=FkSp8VW8RjH.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020. URL https://ojs.aaai.org/index.php/AAAI/article/view/6399.

Elizabeth Salesky, David Etter, and Matt Post. Robust open-vocabulary translation from visual text representations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021. URL https://aclanthology.org/2021.emnlp-main.576.

Mike Schuster and Kaisuke Nakajima. Japanese and korean voice search. In *2012 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, 2012.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2016. URL https://aclanthology.org/P16-1162.

Kevin Slagle. Spacebyte: Towards deleting tokenization from large language modeling. *arXiv preprint arXiv:2404.14408*, 2024.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 2018.

Ilya Sutskever, James Martens, and Geoffrey E Hinton. Generating text with recurrent neural networks. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pages 1017–1024, 2011.

Shawn Tan, Yikang Shen, Zhenfang Chen, Aaron Courville, and Chuang Gan. Sparse universal transformer. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://openreview.net/forum?id=yXYJPAlLqn.

Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=JtBRnrlOEFN.

Mathurin Videau, Badr Youbi Idrissi, Alessandro Leite, Marc Schoenauer, Olivier Teytaud, and David Lopez-Paz. From bytes to ideas: Language modeling with autoregressive u-nets. *arXiv preprint arXiv:2506.14761*, 2025.

Junxiong Wang, Tushaar Gangavarapu, Jing Nathan Yan, and Alexander M Rush. Mambabyte: Token-free selective state space model. *arXiv preprint arXiv:2401.13660*, 2024.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel loop transformer for efficient test-time computation scaling. *arXiv preprint arXiv:2510.24824*, 2025a.

Bohong Wu, Shen Yan, Sijun Zhang, Jianqiao Lu, Yutao Zeng, Ya Wang, and Xun Zhou. Efficient pretraining length scaling. *arXiv preprint arXiv:2504.14992*, 2025b.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 2022. URL https://aclanthology.org/2022.tacl-1.17.

Jin Yang, Zhiqiang Wang, Yanbin Lin, and Zunduo Zhao. Problematic tokens: Tokenizer bias in large language models. *arXiv preprint arXiv:2406.11214*, 2024.

Da Yu, Edith Cohen, Badih Ghazi, Yangsibo Huang, Pritish Kamath, Ravi Kumar, Daogao Liu, and Chiyuan Zhang. Scaling embedding layers in language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=gH4BRa4ZP3.

Lili Yu, Daniel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=JTmO2V9Xpz.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019. URL https://aclanthology.org/P19-1472.

Boyi Zeng, Shixiang Song, Siyuan Huang, Yixuan Wang, He Li, Ziwei He, Xinbing Wang, Zhiyu li, and Zhouhan Lin. PonderLM: Pretraining language models to ponder in continuous space. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=UrM4MNRYZm.

Lin Zheng, Chong Wang, and Lingpeng Kong. Linear complexity randomized self-attention mechanism. In *Proceedings of the 39th International Conference on Machine Learning*, 2022. URL https://proceedings.mlr.press/v162/zheng22b.html.

Lin Zheng, Jianbo Yuan, Chong Wang, and Lingpeng Kong. Efficient attention via control variates. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=G-uNfHKrj46.

Lin Zheng, Xueliang Zhao, Guangtao Wang, Chen Wu, David Dong, Angela Wang, Mingran Wang, Yun Du, Haige Bo, Amol Sharma, Bo Li, Kejie Zhang, Changran Hu, Urmish Thakker, and Lingpeng Kong. Evabyte: Efficient byte-level language models at scale, 2025. URL https://hkunlp.github.io/blog/2025/evabyte.

Lin Zheng, Xinyu Li, Qian Liu, Xiachong Feng, and Lingpeng Kong. Proxy compression for language modeling. *arXiv preprint arXiv:2602.04289*, 2026.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025.

Table 3: Model hyperparameters across architectures. All configurations are designed to yield roughly the same total parameter count. For patch-based byte-level models, the encoder and decoder are lightweight Transformer stacks operating at byte-level resolution, while the main trunk operates on the patch-level sequence; its sequence length varies across runs depending on the realized patch size and scratchpads.

Hyperparameter	**Byte-level**	**Tokenizer-based**	**Patch-based Models**		
			Encoder	Main Trunk	Decoder
Sequence length	8192	2216	8192	—	8192
Hidden dim (d_{model})	2048	2048	1024	2048	1024
FFN hidden size (d_{ff})	16384	16384	8192	16384	8192
Num. layers (L)	18	16	4	16	4
Vocab size ($\|V\|$)	320	100864	320	—	—

A Model Architecture Details

This section provides a comprehensive description of the model architectures in our experiments. All models are designed to have roughly the same total parameter count, enabling fair comparison across different modeling paradigms. We consider three families of models: (i) a **byte-level Transformer** that operates directly on UTF-8 bytes without patching or tokenization, (ii) a **tokenizer-based Transformer** that operates on subword tokens, and (iii) **patch-based byte-level models** that group bytes into patches before processing them through a main trunk. All Transformer layers use a pre-norm design with standard multi-head attention and GEGLU feed-forward networks. Table 3 summarizes key hyperparameters for each model family.

Byte-level Transformer. The byte-level baseline is a standard autoregressive Transformer that predicts the next byte given all preceding bytes, without patching or tokenization. It uses a vocabulary of size 320, consisting of 256 UTF-8 byte values plus 64 reserved sentinel tokens (e.g., `<bos>` and `<pad>`). Because this embedding table and the output head are substantially smaller than those of a tokenizer-based model, we compensate by using additional Transformer layers (18 instead of 16) to keep the total parameter count comparable.

Tokenizer-based Transformer. The tokenizer-based baseline follows a standard autoregressive Transformer architecture operating on subword tokens. The tokenizer has a vocabulary of size $100{,}864$ and is trained on a subsampled corpus from the training set, achieving an average of 3.7 bytes per token and reflecting the substantial share of code data in our training mixture. The larger embedding and language modeling head account for a significant fraction of the total parameters.

Patch-based Byte-level Models. All patch-based architectures share the five-component design described in Section 2: an encoder $\mathcal{E}$, a patchifier $\mathcal{P}$, a main trunk $\mathcal{M}$, an unpatchifier $\mathcal{U}$, and a decoder $\mathcal{D}$. They share the same byte-level vocabulary of size 320 as the byte-level baseline. The encoder and decoder are lightweight Transformer stacks (4 layers each, $d_{\text{model}} = 1024$, $d_{\text{ff}} = 8192$) operating at byte-level resolution, while the main trunk, which accounts for the majority of model parameters and compute, matches the dimensionality of the tokenizer-based model ($d_{\text{model}} = 2048$, $d_{\text{ff}} = 16{,}384$, $L = 16$) and processes the patch-level sequence.

We use full self-attention layers for the encoder $\mathcal{E}$, the trunk $\mathcal{M}$, and decoder $\mathcal{D}$. We observed a slight regression in validation BPB when replacing them with sliding-window attention, though the difference is not significant enough. We leave more efficient encoder and decoder layer designs as future work; alternatives such as Mamba [Gu and Dao, 2024, Dao and Gu, 2024, Lahoti et al., 2026] and xLSTM [Beck et al., 2024] are also viable and might offer a better trade-off between compute and performance, as demonstrated in recent work [Pagnoni et al., 2024, Hwang et al., 2025, Minixhofer et al., 2025].

Different patch-based baselines, including fixed-size patching, SpaceByte, entropy-based patching, and H-Net models, differ only in their patchifier implementation, except for H-Net, which additionally modifies the unpatchifier. Full implementation details are provided in the next section.

B Implementation Details

This section consolidates the implementation specifics for patch-based byte-level modeling, including patchifier and unpatchifier designs, model variants, and the SP configuration.

B.1 Patchifier and Unpatchifier Design

The patchifier $\mathcal{P}$ partitions the byte sequence into L contiguous segments $[s_\ell, e_\ell]$ for each $\ell \in \{1, \ldots, L\}$. It then forms a patch-level representation $z_\ell \coloneqq \text{Aggregate}\,(x_{s_\ell:e_\ell})$ by aggregating all byte hidden states $x_{s_\ell:e_\ell}$. In this work, we implement aggregation as a local cross-attention operation, where a summary vector q_ℓ queries all byte embeddings within the same patch,

$$z_\ell \coloneqq \text{Aggregate}\,(x_{s_\ell:e_\ell}) = \text{CrossAttn}(q_\ell,\ x_{s_\ell:e_\ell},\ x_{s_\ell:e_\ell})\,.$$

q_ℓ is calculated via mean pooling,

$$q_\ell = \frac{1}{e_\ell - s_\ell + 1} \sum_{i=s_\ell}^{e_\ell} x_i.$$

The cross-attention module is multi-headed and masked so that each patch-level summary attends only to byte positions within the corresponding patch. The resulting patch representations are then linearly projected to match the trunk dimensionality.

Symmetrically, trunk outputs pass through the unpatchifier $\mathcal{U}$, which broadcasts them back to byte positions, and are also linearly projected to the decoder dimensionality and fused with the encoder residual.

We conducted ablation studies on two design choices:

Cross-attention. Following Pagnoni et al. [2024], we investigated adding cross-attention at the patchifier (patch-level queries attending to byte-level keys/values) and at the unpatchifier (byte-level queries attending to split patch-level keys/values). We found that cross-attention at the patchifier yields noticeable BPB improvements, whereas adding it at the unpatchifier provides no measurable benefit. We therefore retain cross-attention only at the patchifier in all experiments.

Pooling Strategy. Our default patchifier computes a mean-pooled summary over the byte-level hidden states within each patch, which then serves as the query in the cross-attention mechanism described above. We compared this against a take-last baseline [Hwang et al., 2025] that uses the final byte-level hidden state of each patch as the query. Mean pooling yields consistent but small improvements; we adopt it as our default.

B.2 Patch-based Model Variants

Fixed-size Patching. Following previous practices [Nawrot et al., 2022, Yu et al., 2023], we group bytes into non-overlapping chunks of a fixed width $p \in \{2, 4, 8, 16\}$.

SpaceByte Patching. Following Slagle [2024], we use the same delimiter heuristic that ends patches at whitespace-like boundaries, producing variable-length patches with an average of roughly 6.3 bytes per patch on our training corpus.

Entropy-based Patching. We allocate two additional Transformer layers (with the same configuration as the encoder layer) on top of the encoder $\mathcal{E}$ output, followed by an auxiliary language modeling head that predicts the next byte. The entropy of this prediction is computed at each byte position, and positions where the entropy exceeds a threshold τ_P are marked as patch boundaries, concentrating boundaries in information-dense regions. This auxiliary head is trained jointly with the main LM head in decoder $\mathcal{D}$ by summing their losses with equal weight. We stop the gradient from the auxiliary head to the encoder to prevent it from influencing the encoder representations. In our work, we experiment with thresholds $\tau_\text{P} \in \{1.5, 2.5, 3\}$ and found $\tau_\text{P} = 2.5$ yields the best quality-efficiency trade-off.

H-Net Patching. We closely follow the single-stage formulation of H-Net [Hwang et al., 2025] and implement it upon our shared architecture backbone. Similar to entropy-based patching, we add two Transformer layers on top of the encoder output. We replace the cosine-similarity scoring of the original design [Hwang et al., 2025] with a 2-layer MLP applied to the concatenation of the current and previous byte-level hidden states, which we find to be slightly more stable and performant during training. Patch boundaries are determined by thresholding the MLP sigmoid scores at 0.5, supervised by an auxiliary ratio loss with weight 0.03 as in Hwang et al. [2025] that encourages convergence to a target average patch size $C = 6$. We also trained a variant targeting $C = 3$ for Figs. 5 and 11. We incorporate the smoothing operation from H-Net at the unpatchifier, which improves BPB but slows convergence to the targeted patch size; without smoothing, the model reaches the target faster but with quality degradation. We also experimented with the straight-through confidence-weighted decompression [Hwang et al., 2025], and found it to have marginal effects. We do not use module-wise learning rate modulations.

B.3 Scratchpad Patching Configuration

Unless stated otherwise, scratchpad updates are triggered by an entropy-based policy: a scratchpad update is issued at byte position n whenever the encoder's next-byte prediction entropy H_n exceeds a threshold τ_{SP}. We tune τ_{SP} over $\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$ per patchifier family for the best quality-efficiency trade-off, settling on $\tau_{\text{SP}} = 1.5$ for fixed-size and SpaceByte, $\tau_{\text{SP}} = 1.0$ for entropy-based, and $\tau_{\text{SP}} = 2.5$ for H-Net patching. The higher threshold for H-Net is due to the offset-by-one coupling between scratchpad updates and router-induced patch boundaries discussed in Section E.3: frequent updates (i.e., a lower threshold) introduce redundant computation without further benefit. Conversely, entropy-based patching admits a lower τ_{SP} because both patchification and scratchpad triggering derive from the same entropy signal under $\tau_{\text{P}} > \tau_{\text{SP}}$, so scratchpads fill moderate-entropy gaps without redundantly clustering near boundaries. At most one scratchpad update is applied per byte position; extending this to multiple updates per position is a natural direction for future work.

Entropy prediction heads. Similar to entropy-based patching, the entropy signal is obtained from an auxiliary language modeling head built on two additional Transformer layers placed on top of the encoder, with gradients stopped at the encoder output. This auxiliary head is trained jointly with the main LM head in decoder $\mathcal{D}$ by summing their losses with equal weight.

For fixed-size and SpaceByte patching, these two layers and the output head are allocated specifically for SP. For entropy-based patching, we reuse the existing auxiliary head (already present for patch boundary decisions), maintaining two separate thresholds $\tau_{\text{P}} > \tau_{\text{SP}}$ for both patch boundaries and scratchpad updates. For H-Net, we reuse the two added Transformer layers but allocate a separate language modeling head for entropy prediction.

Design choices. We experimented with several configurations of the auxiliary entropy predictor. Stopping gradients from the auxiliary head to the encoder consistently outperforms the variant without gradient stopping. Using two Transformer layers substantially outperforms a single layer or a direct output head on top of the encoder without additional layers; however, adding more layers or increasing their capacity does not yield further improvements. We observe the same trends for the entropy-based and H-Net patch boundary predictors as well.

C Training Details

C.1 Training Data

All models are pretrained on a mixture of open-source datasets, consisting of DCLM [Li et al., 2024], Stack v2 [Lozhkov et al., 2024], and OpenWebMath [Paster et al., 2024], spanning code, natural language, and mathematics.

C.2 Training Hyperparameters

Comparing models with different input representations involves several confounding factors: the number of unique bytes seen, total FLOPs, parameter count, effective context length, and training schedule. In this work, we fix the number of raw bytes seen across all models to ensure *fixed-data*

comparison, reflecting a data-bounded training regime. Because patch-based models shorten the sequence processed by the main trunk, and SP introduces additional refinement steps, total training FLOPs vary across methods. All other optimization hyperparameters are held constant across models unless otherwise specified.

All models have roughly 2B parameters and are trained for $50{,}000$ steps with a batch size of 8M *bytes* on TPUs, totaling approximately 400B bytes of training data. The context length is set to $2{,}216$ tokens for the tokenizer-based model (corresponding to roughly $8{,}192$ bytes at the observed patch size) and $8{,}192$ bytes for standard and patch-based byte-level models, all with a sequence batch size of $1{,}024$. We use AdamW [Loshchilov and Hutter, 2019] with a learning rate of 1×10^{-3}, $2{,}000$ warmup steps, and cosine decay to 10% of the peak value. We set AdamW's $\epsilon = 1 \times 10^{-12}$, which we found to outperform larger values; the gain is most pronounced for models operating on byte inputs, including both the pure byte-level baseline and patch-based variants, and noticeably smaller for the tokenizer-based model. All input sequences are prepended with a `<bos>` sentinel token.

D Evaluation Details

We evaluate all models on (i) validation Bits-Per-Byte (BPB), (ii) code generation performance, and (iii) multiple-choice natural language understanding tasks.

Validation BPB. We report BPB on a held-out validation split of the training corpus. To enable finer-grained analysis, we maintain separate validation splits for code, natural language, and math data.

Code Generation. We evaluate on MBPP [Austin et al., 2021] and HumanEval [Chen et al., 2021] benchmarks, reporting pass@1 estimated from 5 samples per problem using the standard unbiased estimator of Chen et al. [2021] with $n = 5$ and $k = 1$, under a fixed decoding configuration: temperature 0.2 with nucleus sampling ($p = 0.95$). We use 3-shot prompting with the standard prompts from MBPP and zero-shot prompting for HumanEval.

Natural Language Understanding. We evaluate on a suite of eight multiple-choice downstream benchmarks spanning commonsense reasoning, reading comprehension, and broad knowledge: **ARC-Easy and ARC-Challenge** [Clark et al., 2018], **BoolQ** [Clark et al., 2019], **HellaSwag** [Zellers et al., 2019], **OpenBookQA (OBQA)** [Mihaylov et al., 2018], **PIQA** [Bisk et al., 2020], **WinoGrande** [Sakaguchi et al., 2020], and **MMLU** [Hendrycks et al., 2021]. For all tasks, we follow the OLMES evaluation protocol [Gu et al., 2024] with curated 5-shot prompting.

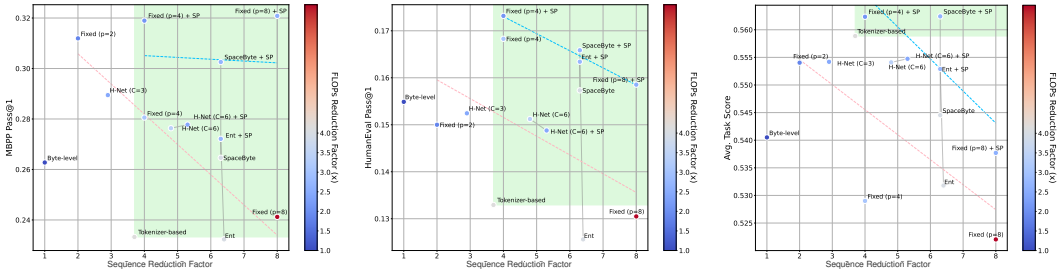
E Additional Experimental Results

E.1 Downstream Quality-Efficiency Pareto Frontiers

The main text (Fig. 5) establishes that SP shifts the validation BPB quality-efficiency frontier. In this section, we verify that this improvement carries over to downstream task performance. Figs. 11a and 11b plot pass@1 rates on MBPP and HumanEval against sequence reduction, while Fig. 11c demonstrates the average accuracy over the eight natural language benchmarks. Across all these settings, SP variants consistently achieve higher downstream task performance at a given sequence reduction factor, confirming that the BPB gains from SP translate into task-level improvements.

E.2 Ablations of Scratchpad Patching Strategies

We ablate the triggering strategy used for scratchpad updates, using fixed-size patching with patch size $p = 8$ as the base patchifier. Our default policy issues a scratchpad update whenever the encoder's next-byte prediction entropy exceeds a threshold, where we use $\tau_{\text{SP}} = 1.5$ by default. We compare against alternative strategies: entropy thresholds (denoted as $E > \tau_{\text{SP}}$) with $\tau_{\text{SP}} \in \{0, 1, 1.5, 2, 2.5, 8\}$, fixed-stride updates every $S \in \{1, 2, 4, 8\}$ positions, and updates on whitespace-like bytes. All configurations are trained under the same setup as the default model. The two extremes are instructive: $\tau_{\text{SP}} = 8$ or stride $S = 8$ effectively suppresses all scratchpads, recovering the non-SP baseline; $\tau_{\text{SP}} = 0$ or $S = 1$ applies a scratchpad update at every byte position, matching the compute of a

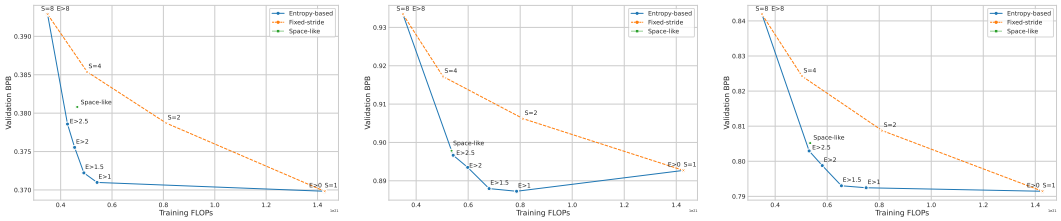


(a) MBPP pass@1.

(b) HumanEval pass@1.

(c) NL benchmark average accuracy.

Figure 11: **Downstream performance versus sequence reduction factor.** The factor is measured as average bytes per persistent model element; larger values indicate fewer trunk/KV-cache states per byte. SP consistently shifts the Pareto frontier across code generation and natural language understanding tasks, confirming that BPB gains translate to downstream improvements. Both the FLOPs coloring and sequence reduction factor are measured under the training configuration; for inference-time FLOPs and KV-cache reduction on downstream tasks, see Tables 1 and 2.



(a) Validation BPB on code.

(b) Validation BPB on natural language.

(c) Validation BPB on math data.

Figure 12: **Ablations of scratchpad triggering strategies** on validation BPB versus training FLOPs, using fixed-size patching ($p = 8$) as the base patchifier. Entropy-based triggers ($E > \tau_{\text{SP}}$), fixed-stride updates (S), and whitespace-based heuristics are compared. The top-left point ($\tau_{\text{SP}} = 8$ or $S = 8$) corresponds to the non-SP baseline; the bottom-right ($\tau_{\text{SP}} = 0$ or $S = 1$) applies dense byte-level compute while *retaining the same committed patch sequence*.

byte-level model. Importantly, this setting is *not* equivalent to a standard byte-level baseline: the model still retains the same number of persistent patches for the KV cache, but allocates byte-level compute via transient scratchpad states. This highlights a unique advantage of SP, which decouples compute allocation from the committed patch sequence, enabling byte-level compute with patch-level memory efficiency.

Fig. 12a reveals several findings. First, entropy-based triggering achieves the best compute-quality trade-off across all three domains, and all strategies, including simple whitespace-based heuristics, improve consistently over the non-SP baseline, confirming that within-patch scratchpads are beneficial regardless of the triggering policy.

Second, the three domains exhibit a revealing difference in how they respond to dense updates. On code and math (Figs. 12a and 12c), the gap between $\tau_{\text{SP}} = 1.0$ and $\tau_{\text{SP}} = 0$ (every-position updates) is marginal, despite the latter requiring roughly $3\times$ the training FLOPs. This indicates that selective, entropy-guided scheduling captures most of the benefit of dense refinement at a fraction of the cost, and further compute yields diminishing returns. On natural language (Fig. 12b), however, dense updates yield slightly *worse* BPB than selective thresholds such as $\tau_{\text{SP}} = 1.0$ or 1.5. We hypothesize that this is related to the entropy profile of natural language: the majority of byte positions in prose are highly predictable (e.g., completing common words), and forcing the model to re-process these trivial positions can dilute the training signal and cause the model to overfit to local patterns at the expense of long-range dependencies. Code and math, by contrast, usually have more uniformly distributed entropy across positions (e.g., syntax, operators, variable names, and numerical expressions are less locally predictable), so dense updates may waste less capacity on trivial bytes and continue to yield

modest gains. This confirms that selective, entropy-guided scheduling is not only more efficient but can be actively preferable to dense refinement.

E.3 Patchification Behavior

While the main text illustrates SP on fixed-size patching (Fig. 3), here we qualitatively examine how SP interacts with the other three patchifier families. The H-Net patchifier (Fig. 13) exhibits strong spatial coupling between scratchpad updates and patch boundaries, with updates often offset by a single position. This leads to redundant computation on adjacent bytes, which is typically unnecessary due to strong local correlations, and helps explain why SP yields smaller gains (and occasionally inefficiencies) for such patchifiers.

SP also integrates naturally with other patchifier families. For SpaceByte (Fig. 15), scratchpad updates frequently coincide with patch boundaries; since patchification takes precedence in such cases (Section 3.1), no redundant computation is introduced. For entropy-based patching (Fig. 14), scratchpads and patch boundaries are coordinated through distinct thresholds, ensuring well-separated compute allocation by design. These qualitative patterns are consistent with the quantitative results in earlier sections and underscore that compatibility between patchification and scratchpad scheduling is critical for effective compute use.

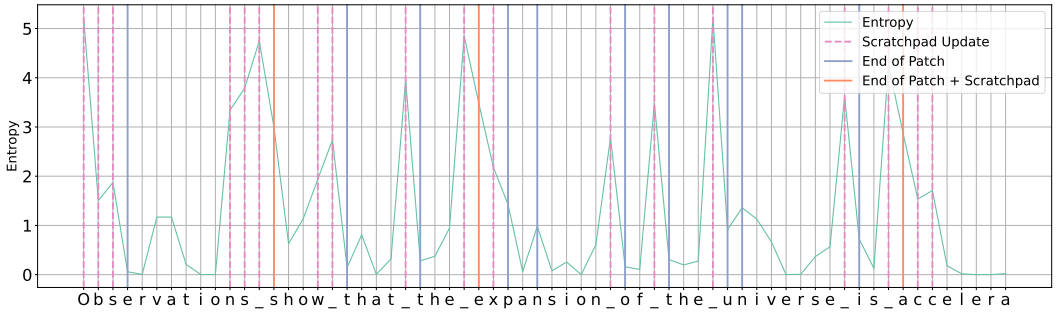


Figure 13: **Scratchpad Patching dynamics on H-Net patching.** Patch boundaries (**solid blue**) are determined by the learned score, while scratchpad updates (**dashed pink**) fire when the encoder's next-byte entropy (**green**) exceeds the threshold. When a scratchpad-trigger coincides with a patch boundary, patchification takes precedence (**solid orange**). We use $\tau_{\text{SP}} = 1.5$ in this demo (experiments use $\tau_{\text{SP}} = 2.5$; Section B.3) to highlight the strong spatial coupling between scratchpad triggers and patch boundaries: scratchpad triggers frequently fall one position before a patch boundary, producing redundant compute on adjacent bytes that are already locally correlated.

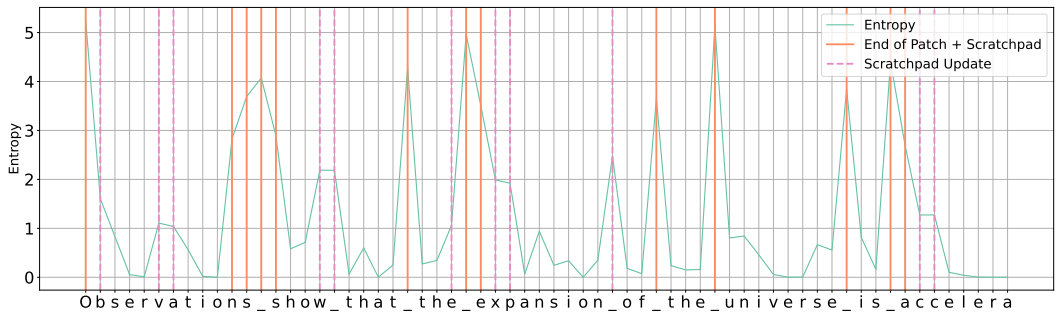


Figure 14: **Scratchpad Patching dynamics on entropy-based patching.** Patch boundaries (**solid blue**) are placed where the encoder's next-byte entropy (**green**) exceeds the patching threshold $\tau_{\text{P}} = 2.5$, while scratchpad updates (**dashed pink**) fire with the lower threshold $\tau_{\text{SP}} = 1.0$. When a scratchpad-trigger coincides with a patch boundary, patchification takes precedence (**solid orange**).

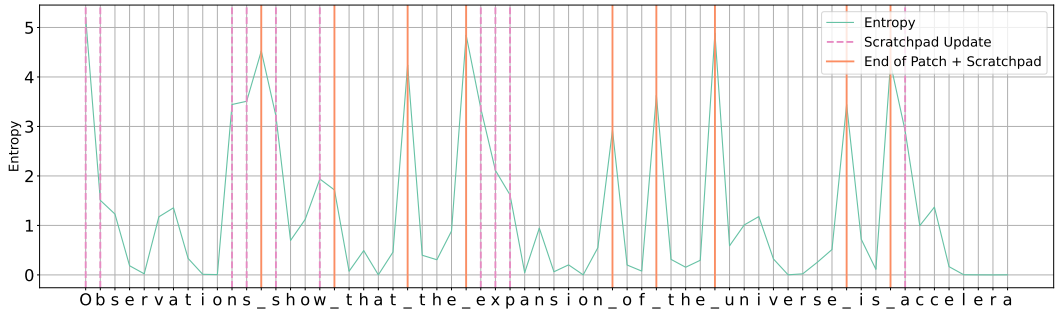


Figure 15: **Scratchpad Patching dynamics on SpaceByte patching.** Patch boundaries (**solid blue**) are placed at whitespace-like delimiters, while scratchpad updates (**dashed pink**) fire when the encoder's next-byte entropy (**green**) exceeds threshold $\tau_{\text{SP}} = 1.5$. When a scratchpad-trigger coincides with a patch boundary, patchification takes precedence (**solid orange**).